PRM Tool — All Architecture Diagrams (Mermaid Source)

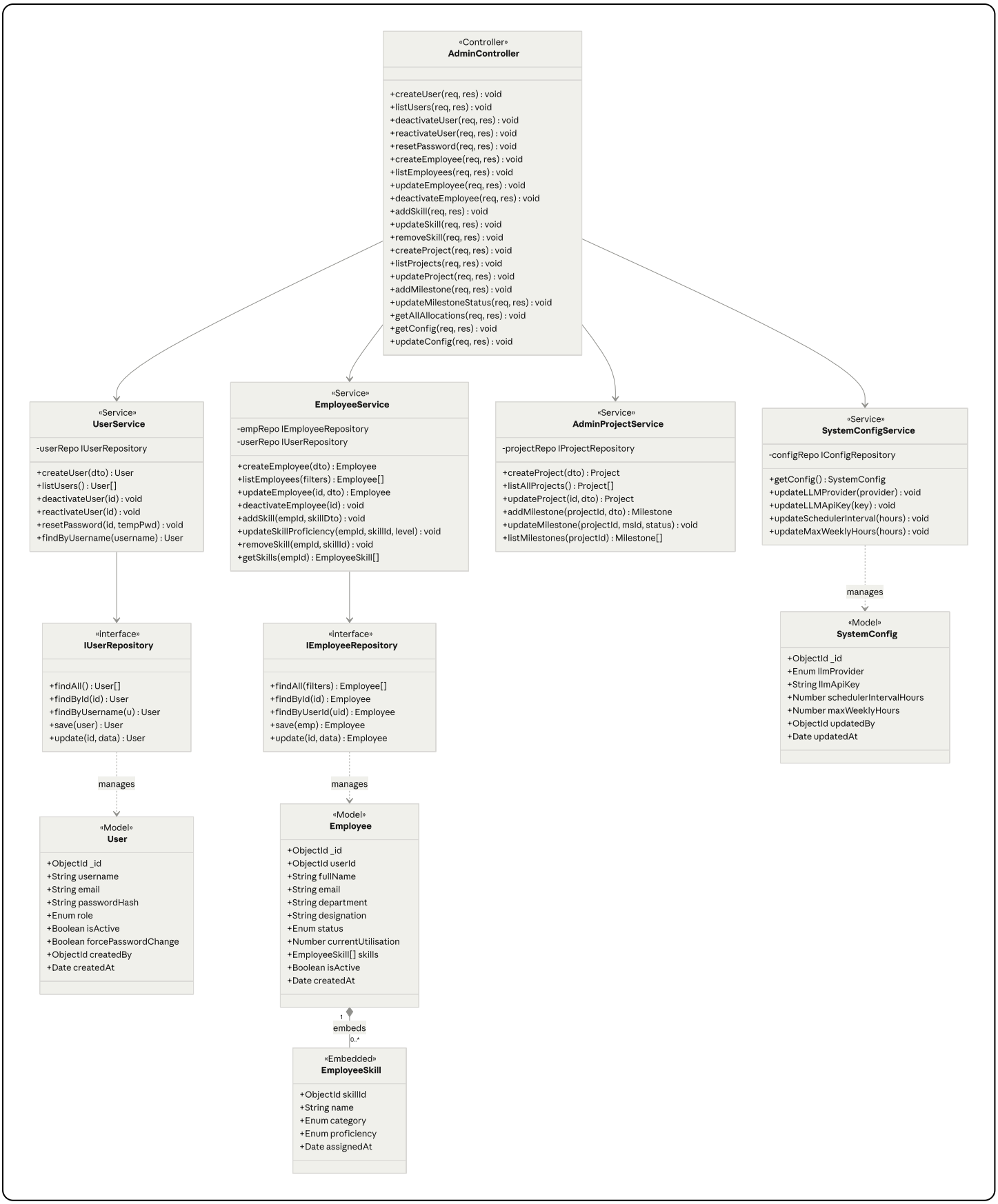
> **Copy any block below into Mermaid Live Editor to render it.**

Table of Contents

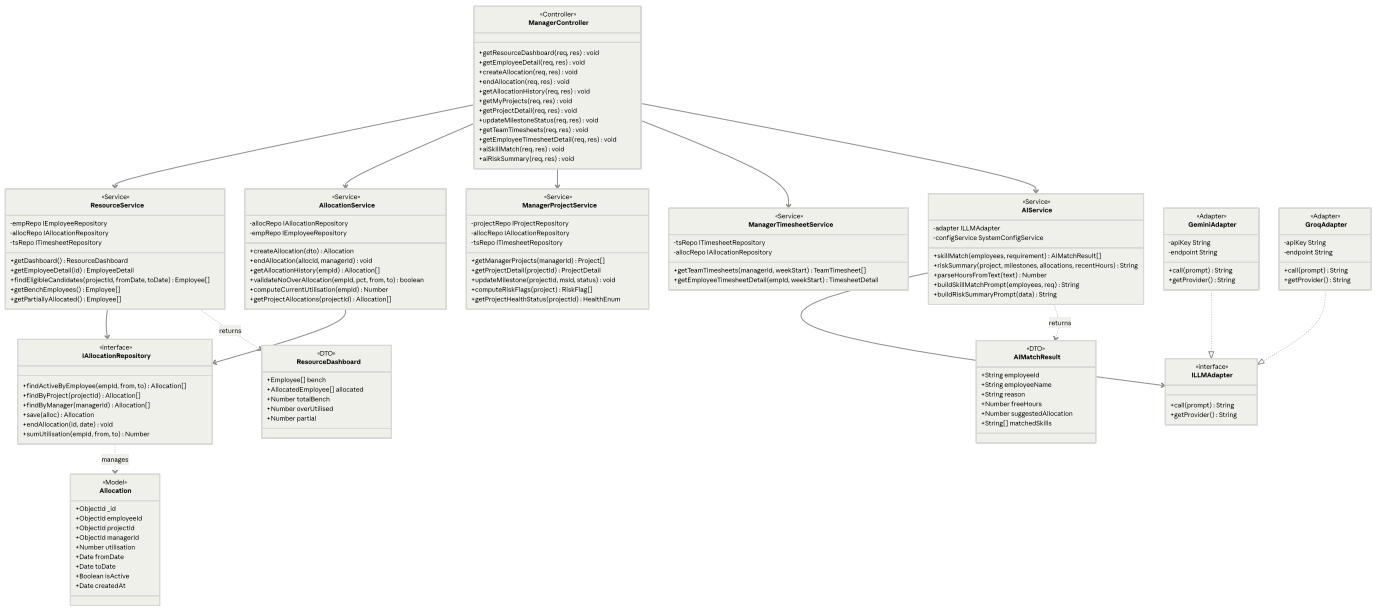
---

1. Class Diagrams

1A — Admin Role Class Diagram



1B — Manager Role Class Diagram



1C — Employee Role Class Diagram

```
classDiagram
  direction TB

  class EmployeeController {
    <<Controller>>
    +getMyAllocations(req, res) void
    +getMyTimesheets(req, res) void
    +getTimesheetDetail(req, res) void
    +checkMissedWeek(req, res) void
    +getActiveAllocationsForWeek(req, res) void
    +submitTimesheet(req, res) void
  }

  class EmployeeTimesheetService {
    <<Service>>
    -tsRepo ITimesheetRepository
    -allocRepo IAllocationRepository
    -configRepo IConfigRepository
    +submitTimesheet(empId, dto) Timesheet
    +getHistory(empId) Timesheet[]
    +getWeekDetail(empId, weekStart) Timesheet
    +getActiveAllocationsForWeek(empId, weekStart) Allocation[]
    +checkMissedWeek(empId) boolean
    +normaliseWeekStart(date) Date
  }

  class TimesheetValidator {
    <<Validator>>
    -configService SystemConfigService
```

```
  +checkDuplicate(empId, weekStart) boolean
  +checkFutureWeek(weekStart) boolean
  +checkProjectHoursLimit(allocPct, hours, maxWkly) boolean
  +checkTotalHoursLimit(entries, maxWkly) boolean
  +checkEmployeeAllocated(empId, projectId, weekStart) boolean
  +validate(empId, dto) ValidationResult
}

class EmployeeAllocationService {
  <<Service>>
  -allocRepo IAllocationRepository
  +getMyAllocations(empId) Allocation[]
  +getActiveAllocations(empId) Allocation[]
  +getCurrentUtilisation(empId) Number
}

class ITimesheetRepository {
  <<interface>>
  +findByEmployee(empId) Timesheet[]
  +findByEmployeeAndWeek(empId, weekStart) Timesheet
  +findByProjectAndWeek(projectId, weekStart) Timesheet[]
  +save(ts) Timesheet
  +insertMissed(empId, weekStart) void
}

class Timesheet {
  <<Model>>
  +ObjectId _id
  +ObjectId employeeId
  +Date weekStart
  +Enum status
  +Number totalHours
  +TimesheetEntry[] entries
  +Date submittedAt
}

class TimesheetEntry {
  <<Embedded>>
  +ObjectId projectId
  +Number hoursLogged
  +String[] activityTags
}

class TimesheetSubmitDTO {
  <<DTO>>
  +Date weekStart
  +TimesheetEntryDTO[] entries
}

class TimesheetEntryDTO {
  <<DTO>>
  +String projectId
  +Number hours
```

```
    +String[] activityTags
  }

  class ValidationResult {
    <<DTO>>
    +Boolean isValid
    +String[] errors
  }

  EmployeeController --> EmployeeTimesheetService
  EmployeeController --> EmployeeAllocationService
  EmployeeTimesheetService --> TimesheetValidator
  EmployeeTimesheetService --> ITimesheetRepository
  TimesheetValidator ..> ValidationResult : returns
  ITimesheetRepository ..> Timesheet : manages
  Timesheet "1" *-- "1..*" TimesheetEntry : embeds
  EmployeeController ..> TimesheetSubmitDTO : receives
  TimesheetSubmitDTO "1" *-- "1..*" TimesheetEntryDTO : contains
```

1D — Full Application Class Diagram

```
classDiagram
  direction TB

  %% ─── Core Models ──────────────────────────────────────────
  class User {
    <<Model>>
    +ObjectId _id
    +String username
    +String email
    +String passwordHash
    +Enum role
    +Boolean isActive
    +Boolean forcePasswordChange
    +ObjectId createdBy
    +Date createdAt
  }

  class Employee {
    <<Model>>
    +ObjectId _id
    +ObjectId userId
    +String fullName
    +String department
    +String designation
    +Enum status
    +Number currentUtilisation
    +EmployeeSkill[] skills
    +Boolean isActive
  }
```

```
class Project {
  <<Model>>
  +ObjectId _id
  +String name
  +String description
  +ObjectId managerId
  +Date startDate
  +Date endDate
  +Enum status
  +Enum healthStatus
  +Milestone[] milestones
  +Date createdAt
}

class Allocation {
  <<Model>>
  +ObjectId _id
  +ObjectId employeeId
  +ObjectId projectId
  +ObjectId managerId
  +Number utilisation
  +Date fromDate
  +Date toDate
  +Boolean isActive
}

class Timesheet {
  <<Model>>
  +ObjectId _id
  +ObjectId employeeId
  +Date weekStart
  +Enum status
  +Number totalHours
  +TimesheetEntry[] entries
  +Date submittedAt
}

class SystemConfig {
  <<Model>>
  +ObjectId _id
  +Enum llmProvider
  +String llmApiKey
  +Number schedulerIntervalHours
  +Number maxWeeklyHours
  +ObjectId updatedBy
}

class Milestone {
  <<Embedded>>
  +String title
  +Date dueDate
  +Enum status
```

```
}

class EmployeeSkill {
  <<Embedded>>
  +String name
  +Enum category
  +Enum proficiency
}

class TimesheetEntry {
  <<Embedded>>
  +ObjectId projectId
  +Number hoursLogged
  +String[] activityTags
}

%% ── Services ──────────────────────────────────────────
class AuthService {
  <<Service>>
  +login(username, password) TokenPayload
  +changePassword(userId, newPwd) void
  +logout(token) void
  +verifyToken(token) Payload
}

class UserService {
  <<Service>>
  +createUser(dto) User
  +listUsers() User[]
  +deactivateUser(id) void
  +reactivateUser(id) void
  +resetPassword(id, pwd) void
}

class EmployeeService {
  <<Service>>
  +createEmployee(dto) Employee
  +updateEmployee(id, dto) Employee
  +deactivateEmployee(id) void
  +manageSkills(empId, op, data) void
}

class ProjectService {
  <<Service>>
  +createProject(dto) Project
  +updateProject(id, dto) Project
  +manageMilestone(projectId, op, data) void
  +getProjectsForManager(managerId) Project[]
  +computeHealthStatus(projectId) HealthEnum
}

class AllocationService {
  <<Service>>
```

```
  +createAllocation(dto) Allocation
  +endAllocation(id, managerId) void
  +validateNoOverAllocation(empId, pct, from, to) boolean
  +computeCurrentUtilisation(empId) Number
}

class TimesheetService {
  <<Service>>
  +submitTimesheet(empId, dto) Timesheet
  +getHistory(empId) Timesheet[]
  +getTeamTimesheets(managerId, week) TeamTimesheet[]
  +validateSubmission(empId, dto) ValidationResult
}

class AIService {
  <<Service>>
  -adapter ILLMAdapter
  +skillMatch(employees, requirement) AIMatchResult[]
  +riskSummary(project, data) String
}

class SchedulerService {
  <<Service>>
  -cronJob CronJob
  +start(intervalHours) void
  +stop() void
  +computeUtilisation() void
  +flagProjectHealth() void
  +markMissedTimesheets() void
}

%% ── LLM Adapter Pattern ──────────────────────────────
class ILLMAdapter {
  <<interface>>
  +call(prompt) String
  +getProvider() String
}

class GeminiAdapter {
  +call(prompt) String
  +getProvider() String
}

class GroqAdapter {
  +call(prompt) String
  +getProvider() String
}

%% ── Auth Middleware ──────────────────────────────────
class AuthMiddleware {
  <<Middleware>>
  +verifyToken(req, res, next) void
  +requireRole(role) Middleware
```

```
    +checkForcePasswordChange(req, res, next) void
  }

  %% ── Model Relationships ──────────────────────────────
  User "1" --> "0..1" Employee : has profile
  User "1" --> "0..*" Project : manages
  Employee "1" --> "0..*" Allocation : assigned via
  Project "1" --> "0..*" Allocation : has
  Project "1" *-- "0..*" Milestone : contains
  Employee "1" *-- "0..*" EmployeeSkill : has
  Employee "1" --> "0..*" Timesheet : submits
  Timesheet "1" *-- "1..*" TimesheetEntry : contains

  %% ── Service Relationships ────────────────────────────
  AuthService --> User : validates
  UserService --> User : manages
  EmployeeService --> Employee : manages
  ProjectService --> Project : manages
  AllocationService --> Allocation : manages
  AllocationService --> Employee : updates status
  TimesheetService --> Timesheet : manages
  AIService --> ILLMAdapter : delegates
  GeminiAdapter ..|> ILLMAdapter
  GroqAdapter ..|> ILLMAdapter
  SchedulerService --> AllocationService : triggers
  SchedulerService --> ProjectService : triggers
  SchedulerService --> TimesheetService : triggers
  AuthMiddleware --> User : checks
```

---

2. Use Case Diagrams

> **Note:** Mermaid's native use-case support is limited. The diagrams below use `flowchart` syntax with UML-style annotations to approximate standard use case notation.

2A — Admin Use Case Diagram

```
flowchart LR
  subgraph actors["Actors"]
    A(["👤 Admin"])
  end

  subgraph system["Admin System Boundary"]
    direction TB

    subgraph auth["Authentication"]
      UC1["Login"]
      UC2["Change Password"]
    end
```

```
  subgraph users["User Management"]
    UC3["Create User Account"]
    UC4["View All Users"]
    UC5["Reset User Password"]
    UC6["Deactivate User"]
    UC7["Reactivate User"]
  end

  subgraph emp["Employee Management"]
    UC8["Add Employee Profile"]
    UC9["View All Employees"]
    UC10["Update Employee"]
    UC11["Deactivate Employee"]
    UC12["Manage Employee Skills"]
  end

  subgraph proj["Project Management"]
    UC13["Create Project"]
    UC14["View All Projects"]
    UC15["Update Project"]
    UC16["Add Milestone"]
    UC17["Update Milestone Status"]
  end

  subgraph alloc["Allocation"]
    UC18["View All Allocations\n(Company-wide)"]
  end

  subgraph cfg["System Config"]
    UC19["Configure LLM Provider & Key"]
    UC20["Set Scheduler Interval"]
    UC21["Set Max Weekly Hours"]
  end
end

A --> UC1
A --> UC2
A --> UC3
A --> UC4
A --> UC5
A --> UC6
A --> UC7
A --> UC8
A --> UC9
A --> UC10
A --> UC11
A --> UC12
A --> UC13
A --> UC14
A --> UC15
A --> UC16
A --> UC17
```

```
  A --> UC18
  A --> UC19
  A --> UC20
  A --> UC21

  UC1 -.->|"<<include>>"| UC2
  UC11 -.->|"<<include>>"| UC6
```

2B — Manager Use Case Diagram

```
flowchart LR
  subgraph actors["Actors"]
    M(["🧑‍💼 Manager"])
    AI(["🤖 LLM Service\n(Gemini/Groq)"])
  end

  subgraph system["Manager System Boundary"]
    direction TB

    subgraph auth["Authentication"]
      UC1["Login"]
      UC2["Change Password"]
    end

    subgraph dash["Resource Dashboard"]
      UC3["View Resource Dashboard\n(Bench & Allocated)"]
      UC4["Drill into Employee Detail\n(Skills, Allocations, Tags)"]
    end

    subgraph alloc["Resource Allocation"]
      UC5["AI-Assisted Resource Search"]
      UC6["Direct Allocation\n(Skip AI)"]
      UC7["Confirm Allocation\n(Set %, Dates)"]
      UC8["End Existing Allocation"]
      UC9["View Allocation History"]
    end

    subgraph proj["Project Monitoring"]
      UC10["View My Projects\n(Health Status)"]
      UC11["View Project Detail\n(Milestones, Team, Flags)"]
      UC12["AI Risk Summary"]
    end

    subgraph ts["Timesheets"]
      UC13["View Team Timesheets\n(By Week)"]
      UC14["View Employee Timesheet Detail"]
    end

    subgraph ai["AI Assistant"]
      UC15["Skill Match Query"]
```

```
    UC16["Risk Summary Query"]
  end
end

M --> UC1
M --> UC2
M --> UC3
M --> UC4
M --> UC5
M --> UC6
M --> UC7
M --> UC8
M --> UC9
M --> UC10
M --> UC11
M --> UC12
M --> UC13
M --> UC14
M --> UC15
M --> UC16

UC5 -.->|"<<include>>"| UC7
UC6 -.->|"<<include>>"| UC7
UC11 -.->|"<<extend>>"| UC12
UC13 -.->|"<<extend>>"| UC14
UC15 -.->|"<<include>>"| UC5
UC16 -.->|"<<include>>"| UC12

UC5 --> AI
UC12 --> AI
UC15 --> AI
UC16 --> AI
```

2C — Employee Use Case Diagram

```
flowchart LR
  subgraph actors["Actors"]
    E(["💼 Employee"])
  end

  subgraph system["Employee System Boundary"]
    direction TB

    subgraph auth["Authentication"]
      UC1["Login"]
      UC2["Change Password\n(First Login)"]
    end

    subgraph ts["Timesheet Management"]
      UC3["Submit Weekly Timesheet"]
```

```
      UC4["Select Project (Allocated)"]
      UC5["Enter Hours Per Project"]
      UC6["Select Activity Tags"]
      UC7["View Timesheet History"]
      UC8["View Week Detail"]
    end

    subgraph alloc["Allocation View"]
      UC9["View My Allocations\n(Active & Historical)"]
    end

    subgraph notif["Notifications"]
      UC10["See Missed Timesheet\nReminder Banner"]
    end
  end

  E --> UC1
  E --> UC2
  E --> UC3
  E --> UC7
  E --> UC9

  UC1 -.->|"<<include>>"| UC2
  UC3 -.->|"<<include>>"| UC4
  UC3 -.->|"<<include>>"| UC5
  UC3 -.->|"<<include>>"| UC6
  UC7 -.->|"<<extend>>"| UC8
  UC3 -.->|"<<extend>>"| UC10
```

2D — Full System Use Case Diagram

```
flowchart TB
  subgraph actors_left["Actors"]
    direction TB
    A(["👤 Admin"])
    M(["👔 Manager"])
    E(["💼 Employee"])
    SCH(["⏱️ Background\nScheduler"])
  end

  subgraph system["Project & Resource Management System Boundary"]
    direction TB

    subgraph ADMIN_UC["ADMIN USE CASES"]
      direction LR
      A1["Manage Users\n(Create/Deactivate/Reset)"]
      A2["Manage Employee Profiles\n(Add/Update/Deactivate)"]
      A3["Manage Skills\n(Add/Update/Remove)"]
      A4["Manage Projects\n(Create/Update)"]
      A5["Manage Milestones\n(Add/Update Status)"]
```

```
    A6["View All Allocations"]
    A7["System Configuration\n(LLM Key, Scheduler, Hours)"]
  end

  subgraph MGR_UC["MANAGER USE CASES"]
    direction LR
    M1["Resource Dashboard\n(Bench & Utilisation)"]
    M2["Allocate Resource\n(AI / Direct)"]
    M3["End Allocation"]
    M4["View My Projects\n(Health Status)"]
    M5["AI Risk Summary"]
    M6["View Team Timesheets"]
  end

  subgraph EMP_UC["EMPLOYEE USE CASES"]
    direction LR
    E1["Submit Weekly Timesheet\n(Hours + Activity Tags)"]
    E2["View My Timesheets\n(History & Status)"]
    E3["View My Allocations"]
  end

  subgraph SCHED_UC["AUTOMATED SCHEDULER JOBS"]
    direction LR
    S1["Calculate Utilisation\n(All Employees)"]
    S2["Update Bench Status\n(BENCH / ALLOCATED)"]
    S3["Flag Project Risks\n(Overdue Milestones,\nLow Effort)"]
    S4["Mark Missed Timesheets"]
  end

  UC_AUTH["Authenticate User\n<<include>>"]
  UC_FORCE["Force Password Change\n<<extend>>"]
end

subgraph external["External Services"]
  LLM(["🤖 LLM Service\n(Gemini / Groq)"])
end

A --> A1 & A2 & A3 & A4 & A5 & A6 & A7
M --> M1 & M2 & M3 & M4 & M5 & M6
E --> E1 & E2 & E3
SCH --> S1 & S2 & S3 & S4

A --> UC_AUTH
M --> UC_AUTH
E --> UC_AUTH
UC_AUTH -.->|"<<extend>>"| UC_FORCE

M2 --> LLM
M5 --> LLM

S1 --> S2
```

```
  S1 --> S3
  S1 --> S4
```

---

3. Sequence Diagrams

3A — Login & Force Password Change

```
sequenceDiagram
  autonumber
  actor U as User
  participant FE as React Frontend
  participant MW as Auth Middleware
  participant BE as Auth Controller
  participant DB as MongoDB

  U->>FE: Enter username + password
  FE->>BE: POST /api/auth/login {username, password}
  BE->>DB: findOne({username})
  DB-->>BE: User document (or null)

  alt User not found or inactive
    BE-->>FE: 401 {error: "Invalid credentials"}
    FE-->>U: Show error toast
  else Valid user — wrong password
    BE->>BE: bcrypt.compare(pwd, hash) → false
    BE-->>FE: 401 {error: "Invalid credentials"}
    FE-->>U: Show error toast
  else Valid credentials
    BE->>BE: bcrypt.compare(pwd, hash) → true
    BE->>BE: jwt.sign({userId, role, exp: 8h})
    BE-->>FE: 200 {token, role, forcePasswordChange}

    alt forcePasswordChange == true
      FE->>FE: Store token in memory
      FE->>FE: Redirect → /change-password
      U->>FE: Enter new password + confirm
      FE->>BE: POST /api/auth/change-password {newPassword}
      Note over FE,BE: JWT sent in Authorization header
      BE->>MW: verifyToken(req)
      MW-->>BE: Decoded {userId, role}
      BE->>BE: Validate password strength
      BE->>DB: update({_id: userId}, {passwordHash, forcePasswordChange: false})
      DB-->>BE: Updated
      BE-->>FE: 200 {message: "Password changed"}
      FE->>FE: Redirect → role dashboard
    else forcePasswordChange == false
      FE->>FE: Store token, redirect → role dashboard
    end
```

```
    U-->>FE: Sees their role-specific menu
  end
```

3B — Admin: Create User & Employee Profile

```
sequenceDiagram
  autonumber
  actor ADM as Admin
  participant FE as React Frontend
  participant BE as Express Backend
  participant DB as MongoDB

  Note over ADM,DB: Step 1 — Create User Account
  ADM->>FE: Fill user form (name, email, username, temp pwd, role=EMPLOYEE)
  FE->>BE: POST /api/admin/users
  BE->>DB: findOne({username}) — duplicate check
  DB-->>BE: null (no duplicate)
  BE->>DB: findOne({email}) — email duplicate check
  DB-->>BE: null (no duplicate)
  BE->>BE: bcrypt.hash(tempPassword, 10)
  BE->>DB: insertOne({username, email, passwordHash, role, forcePasswordChange: true, isActi
  DB-->>BE: {_id: "user_001", ...}
  BE-->>FE: 201 {userId: "user_001", message: "Account created"}
  FE-->>ADM: Show success — "Account created, user must change password on login"

  Note over ADM,DB: Step 2 — Create Employee Profile (links to user_001)
  ADM->>FE: Fill employee form (userId="user_001", name, dept, designation)
  FE->>BE: POST /api/admin/employees
  BE->>DB: findOne({_id: "user_001"}) — validate userId exists
  DB-->>BE: User document
  BE->>DB: findOne({userId: "user_001"}) — check no existing profile
  DB-->>BE: null (safe to create)
  BE->>DB: insertOne({userId:"user_001", fullName, department, designation, status:"BENCH",
  DB-->>BE: {_id: "emp_001"}
  BE-->>FE: 201 {employeeId: "emp_001"}
  FE-->>ADM: Show success — "Employee profile created, status: BENCH"

  Note over ADM,DB: Step 3 — Add Skills to Employee
  ADM->>FE: Select employee emp_001, add skill
  FE->>BE: POST /api/admin/employees/emp_001/skills {name:"Java", category:"BACKEND", profic
  BE->>DB: findOne({_id:"emp_001"})
  DB-->>BE: Employee document
  BE->>DB: updateOne({_id:"emp_001"}, {$push: {skills: {name, category, proficiency, assigne
  DB-->>BE: Updated
  BE-->>FE: 200 {message: "Skill added"}
  FE-->>ADM: Skills list refreshed
```

3C — Admin: Create Project & Milestone

```
sequenceDiagram
  autonumber
  actor ADM as Admin
  participant FE as React Frontend
  participant BE as Express Backend
  participant DB as MongoDB

  ADM->>FE: Fill project form (name, description, startDate, endDate, status=ACTIVE, manager
  FE->>BE: POST /api/admin/projects
  BE->>DB: findOne({_id: managerId, role: "MANAGER"}) — validate manager
  DB-->>BE: Manager user document
  BE->>DB: insertOne({name, description, managerId, startDate, endDate, status, healthStatus
  DB-->>BE: {_id: "proj_001"}
  BE-->>FE: 201 {projectId: "proj_001"}
  FE-->>ADM: Project created, show milestone management

  loop Add each Milestone
    ADM->>FE: Enter milestone (title, dueDate)
    FE->>BE: POST /api/admin/projects/proj_001/milestones
    BE->>DB: updateOne({_id:"proj_001"}, {$push: {milestones: {title, dueDate, status:"NOT_S
    DB-->>BE: Updated
    BE-->>FE: 200 Milestone added
    FE-->>ADM: Milestone appears in list
  end

  ADM->>FE: Update milestone status
  FE->>BE: PUT /api/admin/projects/proj_001/milestones/:msId {status:"IN_PROGRESS"}
  BE->>DB: updateOne({_id:"proj_001","milestones._id":msId}, {$set:{"milestones.$.status":"I
  DB-->>BE: Updated
  BE-->>FE: 200 Status updated
```

3D — Manager: AI-Assisted Resource Search & Allocation

```
sequenceDiagram
  autonumber
  actor MGR as Manager
  participant FE as React Frontend
  participant BE as Express Backend
  participant AI as LLM API (Gemini/Groq)
  participant DB as MongoDB

  MGR->>FE: Select project "Alpha Portal", type requirement
  Note right of MGR: "I need a backend developer with\nJava and microservices, 3 months"

  FE->>BE: POST /api/manager/ai/skill-match\n{projectId:"proj_001", requirement:"...", fromD
  BE->>DB: find employees with isActive=true
  DB-->>BE: All active employees (with skills array)
```

```
  BE->>DB: For each employee: sum active allocations in date range
  DB-->>BE: Utilisation map per employee
  BE->>BE: Filter out employees with utilisation >= 100%
  Note over BE: Pre-filter: only eligible candidates passed to LLM

  BE->>AI: POST prompt with candidate summaries + requirement
  Note right of BE: {requirement, candidates:[{name, skills[], freeHours, recentTags[]}]}
  AI-->>BE: Ranked list with reasons (JSON)
  BE->>BE: Parse AI response → AIMatchResult[]
  BE-->>FE: 200 {results: [{name, reason, freeHours, suggestedAllocation, matchedSkills}]}
  FE-->>MGR: Show AI-matched results with reasons

  MGR->>FE: Select employee #1, set utilisation=50%, fromDate, toDate
  FE->>BE: POST /api/manager/allocations\n{employeeId, projectId, utilisation:50, fromDate,
  BE->>DB: Aggregate active allocations for employee in overlapping period
  DB-->>BE: currentTotal = 0%
  BE->>BE: 0% + 50% = 50% ≤ 100% → VALID

  BE->>DB: insertOne(Allocation)
  DB-->>BE: {_id: "alloc_001"}
  BE->>DB: updateOne Employee: currentUtilisation=50, status="ALLOCATED"
  DB-->>BE: Updated
  BE-->>FE: 201 {allocationId: "alloc_001"}
  FE-->>MGR: "Allocation saved ✓"
```

3E — Manager: View Project Health & AI Risk Summary

```
sequenceDiagram
  autonumber
  actor MGR as Manager
  participant FE as React Frontend
  participant BE as Express Backend
  participant AI as LLM API (Gemini/Groq)
  participant DB as MongoDB

  MGR->>FE: Navigate to My Projects
  FE->>BE: GET /api/manager/projects
  BE->>DB: find projects where managerId = currentUser._id
  DB-->>BE: Projects with healthStatus (computed by scheduler)
  BE-->>FE: [{name, endDate, healthStatus: "AT_RISK"}, ...]
  FE-->>MGR: Shows traffic-light health per project

  MGR->>FE: Select "Alpha Portal"
  FE->>BE: GET /api/manager/projects/proj_001
  BE->>DB: findOne Project (with milestones embedded)
  DB-->>BE: Project document
  BE->>DB: find active Allocations for proj_001
  DB-->>BE: Allocation list with employeeIds
  BE->>DB: find Timesheets (last 4 weeks) for those employees on this project
  DB-->>BE: Recent timesheet entries with hours
```

```
  BE->>BE: Compute risk flags:\n  - Milestones overdue?\n  - Hours logged < expected?
  BE-->>FE: {project, milestones, allocations, riskFlags, healthStatus}
  FE-->>MGR: Project detail with risk flags shown

  MGR->>FE: Click "Get AI Risk Summary"
  FE->>BE: POST /api/manager/ai/risk-summary/proj_001
  BE->>DB: Fetch milestones, allocations, last 2 weeks timesheets
  DB-->>BE: Full project health data
  BE->>BE: Build structured risk prompt with facts
  BE->>AI: POST prompt: "Summarise risks for this project in plain English"
  AI-->>BE: Plain-English risk paragraph
  BE-->>FE: 200 {summary: "The Backend API milestone is overdue..."}
  FE-->>MGR: Display AI summary with disclaimer note
```

3F — Manager: View Team Timesheets

```
sequenceDiagram
  autonumber
  actor MGR as Manager
  participant FE as React Frontend
  participant BE as Express Backend
  participant DB as MongoDB

  MGR->>FE: Navigate to Timesheets, enter week date
  FE->>BE: GET /api/manager/timesheets?weekStart=12-05-2026
  BE->>DB: find projects where managerId = currentUserId
  DB-->>BE: Manager's project IDs
  BE->>DB: find active Allocations for those projects (week overlaps)
  DB-->>BE: Allocated employee IDs
  BE->>DB: find Timesheets where employeeId IN [...] AND weekStart = week
  DB-->>BE: Submitted timesheets
  BE->>BE: Cross-reference: employees with allocations but no timesheet → MISSED
  BE-->>FE: [{employeeId, employeeName, projectName, hours, status:SUBMITTED/MISSED}]
  FE-->>MGR: Show team timesheet grid with MISSED warnings

  MGR->>FE: Click employee name to see detail
  FE->>BE: GET /api/manager/timesheets/:empId?weekStart=12-05-2026
  BE->>DB: findOne({employeeId, weekStart})
  DB-->>BE: Timesheet with entries
  BE-->>FE: {entries:[{projectName, hours, activityTags}], totalHours, status}
  FE-->>MGR: Show per-project breakdown with activity tags
```

3G — Employee: Submit Weekly Timesheet

```
sequenceDiagram
  autonumber
  actor EMP as Employee
```

```
  participant FE as React Frontend
  participant BE as Express Backend
  participant DB as MongoDB

  EMP->>FE: Navigate to Submit Timesheet
  FE->>BE: GET /api/employee/timesheets/week-check
  BE->>DB: findOne({employeeId, weekStart: lastMonday})
  DB-->>BE: null (no submission)
  BE-->>FE: {hasMissed: true, weekStart: "06-05-2026"}
  FE-->>EMP: Show reminder banner ⚠

  EMP->>FE: Open Submit Timesheet form, confirm week date
  FE->>BE: GET /api/employee/allocations?weekStart=12-05-2026
  BE->>DB: find allocations where employeeId=me AND fromDate≤weekStart AND toDate≥weekStart
  DB-->>BE: Active allocations for the week
  BE->>DB: find SystemConfig → maxWeeklyHours=40
  DB-->>BE: Config
  BE-->>FE: [{projectId, projectName, utilisation, expectedMaxHours}]
  FE-->>EMP: Show project cards with expected hours

  EMP->>FE: Enter 18hrs for Alpha Portal, tags [Microservices, WebSocket]
  EMP->>FE: Enter 20hrs for Beta CRM, tags [Backend API, Bug Fixing]
  FE->>BE: POST /api/employee/timesheets\n{weekStart, entries:[{projectId, hours, activityTa

  BE->>DB: findOne({employeeId, weekStart}) — duplicate check
  DB-->>BE: null (safe)
  BE->>BE: weekStart > today? → NO (valid)
  BE->>BE: Per project: hours ≤ (utilisation% × maxWeeklyHours)\n  18 ≤ (50% × 40 = 20) ✓\n
  BE->>BE: Total hours: 18+20=38 ≤ 40 ✓

  BE->>DB: insertOne({employeeId, weekStart, status:"SUBMITTED", totalHours:38, entries:[...
  DB-->>BE: {_id: "ts_001"}
  BE-->>FE: 201 {status: "SUBMITTED", totalHours: 38}
  FE-->>EMP: Show success summary with project breakdown
```

3H — Background Scheduler Full Loop

```
sequenceDiagram
  autonumber
  participant CR as node-cron
  participant SC as SchedulerService
  participant DB as MongoDB

  CR->>SC: Trigger (every N hours per SystemConfig)
  SC->>DB: findOne SystemConfig → maxWeeklyHours, schedulerIntervalHours
  DB-->>SC: Config values

  Note over SC,DB: ═══ JOB 1: Compute Utilisation ═══
  SC->>DB: find all Allocations where isActive=true\nAND fromDate ≤ today ≤ toDate
  DB-->>SC: Active allocations [{employeeId, utilisation}]
```

```
  SC->>SC: Group by employeeId, sum utilisation %
  SC->>DB: bulkWrite: update each Employee\n{currentUtilisation: sum, status: BENCH|ALLOCATE
  DB-->>SC: Modified count

  Note over SC,DB: ═══ JOB 2: Flag Project Health ═══
  SC->>DB: find Projects where status = ACTIVE (with milestones)
  DB-->>SC: Active projects
  loop For each project
    SC->>DB: find Allocations for project (isActive=true)
    DB-->>SC: Team allocation list
    SC->>DB: find Timesheets for team, last 2 weeks
    DB-->>SC: Recent hours logged
    SC->>SC: Evaluate health:\n  - Any milestone IN_PROGRESS past dueDate → risk\n  - Hours
    SC->>DB: updateOne Project: {healthStatus: computed}
    DB-->>SC: Updated
  end

  Note over SC,DB: ═══ JOB 3: Mark Missed Timesheets ═══
  SC->>SC: Compute lastCompletedWeekStart (last Monday before today)
  SC->>DB: find Employees with at least 1 active allocation
  DB-->>SC: Active employee IDs
  loop For each employee
    SC->>DB: findOne Timesheet {employeeId, weekStart: lastMonday}
    DB-->>SC: null (no submission)
    SC->>DB: insertOne {employeeId, weekStart: lastMonday, status:"MISSED", totalHours:0, er
    DB-->>SC: Inserted
  end

  SC-->>CR: Jobs complete — next trigger in N hours
```

---

4. Entity Relationship Diagram

```
erDiagram
  USERS {
    ObjectId _id PK
    string username UK
    string email UK
    string passwordHash
    enum role "ADMIN|MANAGER|EMPLOYEE"
    boolean isActive
    boolean forcePasswordChange
    ObjectId createdBy FK
    datetime createdAt
    datetime updatedAt
  }

  EMPLOYEES {
    ObjectId _id PK
    ObjectId userId FK
```

```
  string fullName
  string email
  string department "Backend|Frontend|DevOps|QA|Other"
  string designation
  enum status "BENCH|ALLOCATED|INACTIVE"
  number currentUtilisation "0-100 computed"
  boolean isActive
  datetime createdAt
}

EMPLOYEE_SKILLS {
  ObjectId _id PK
  ObjectId employeeId FK
  string skillName
  enum category "BACKEND|FRONTEND|DEVOPS|QA|OTHER"
  enum proficiency "BEGINNER|INTERMEDIATE|ADVANCED"
  datetime assignedAt
}

PROJECTS {
  ObjectId _id PK
  string name
  string description
  ObjectId managerId FK
  date startDate
  date endDate
  enum status "PLANNED|ACTIVE|ON_HOLD|COMPLETED"
  enum healthStatus "ON_TRACK|ATTENTION|AT_RISK"
  datetime createdAt
}

MILESTONES {
  ObjectId _id PK
  ObjectId projectId FK
  string title
  date dueDate
  enum status "NOT_STARTED|IN_PROGRESS|DONE"
  datetime updatedAt
}

ALLOCATIONS {
  ObjectId _id PK
  ObjectId employeeId FK
  ObjectId projectId FK
  ObjectId managerId FK
  number utilisation "1-100"
  date fromDate
  date toDate
  boolean isActive
  datetime createdAt
}

TIMESHEETS {
```

```
    ObjectId _id PK
    ObjectId employeeId FK
    date weekStart "Always Monday"
    enum status "SUBMITTED|MISSED"
    number totalHours
    datetime submittedAt
  }

  TIMESHEET_ENTRIES {
    ObjectId _id PK
    ObjectId timesheetId FK
    ObjectId projectId FK
    number hoursLogged
    string activityTags "Array"
  }

  SYSTEM_CONFIG {
    ObjectId _id PK
    enum llmProvider "GEMINI|GROQ"
    string llmApiKey "encrypted"
    number schedulerIntervalHours
    number maxWeeklyHours
    ObjectId updatedBy FK
    datetime updatedAt
  }

  USERS ||--o| EMPLOYEES : "has profile"
  USERS ||--o{ PROJECTS : "manages"
  EMPLOYEES ||--o{ EMPLOYEE_SKILLS : "has"
  EMPLOYEES ||--o{ ALLOCATIONS : "allocated via"
  EMPLOYEES ||--o{ TIMESHEETS : "submits"
  PROJECTS ||--o{ MILESTONES : "contains"
  PROJECTS ||--o{ ALLOCATIONS : "receives"
  ALLOCATIONS }o--|| USERS : "created by manager"
  TIMESHEETS ||--o{ TIMESHEET_ENTRIES : "has entries"
  TIMESHEET_ENTRIES }o--|| PROJECTS : "references"
  SYSTEM_CONFIG }o--|| USERS : "updated by"
```

Generated by Senior Technical Architect review of PRM BRD v3 Tech Stack: React + Bootstrap + Tailwind | Node.js + Express.js + JWT | MongoDB